

SwarmSwIM: a simulator for underwater swarms of robots

Saverio Iacoponi ^{*}, Andrea Infanti ^{*†}, Mohammed El Hanbaly ^{*}, Giulia De Masi [‡], Federico Renda ^{*}

^{*}Dept. of Mechanical Eng., Khalifa University, Abu Dhabi, UAE

[†]CNRS - Lab I3S, Université Côte d'Azur, Nice, France

[‡]SAFIR, Dept. of Science and Engineering, Sorbonne University Abu Dhabi, Abu Dhabi, UAE

Abstract—Simulation plays a crucial role in the design, testing, and validation of underwater multi-robot systems, enabling algorithm development in a controlled environment. While high-fidelity dynamic models are essential for certain applications, many multi-robot and swarm control strategies prioritize local control emulation and high-level decision-making over full-scale dynamics. We present SwarmSwIM, a modular simulation platform designed to support a wide range of control schemes and sensor models for underwater multi-robot systems. The simulator provides tools for current modeling, visual representation and rendering, and ROS integration, balancing computational efficiency with sufficient environmental fidelity to facilitate large-scale swarm simulations.

Index Terms—Underwater multi-robot simulator, robotic swarm simulator, multi-agents system

I. INTRODUCTION

The development and testing of underwater robots, particularly Autonomous Underwater Vehicles (AUVs), rely heavily on simulation tools at various stages of the development process. Simulations offer a controlled environment to evaluate and enhance the control systems and decision-making capabilities of underwater agents. While this consideration holds true for most cases in robotics, the underwater environment particularly benefits from simulated testing, as field tests are significantly more challenging to perform, and acquiring ground truth data poses its own set of difficulties. Commonly used simulators in general robotics include Gazebo [1] and RoboDK [2]. Beyond these, several add-ons or standalone simulators are specifically designed for the simulation of marine and underwater robots. For instance, UUV Simulator [3], VRX Simulator [4], UWSim [5], and commercial products such as Dynautics AUV-SIM [6] provide tools for simulating ROVs and AUVs in underwater environments. Such simulations provide virtual environment feedback and often implement full six-degree-of-freedom (6-DoF) dynamics, explicitly simulate thruster outputs, and handle collision events. However, this level of performance often comes at a high computational cost. While the contribution of these simulators to full-loop simulations of one or a few agents is undeniable, this approach becomes less effective when applied to large-scale underwater networks of agents.

A few underwater simulators for multi-robot systems exist [7], such as: UWSim [5], CoppeliaSim [8] (formerly known as V-REP). It is fairly common to see in literature that

specific underwater multi-robot investigations rely on custom-made simulators tailored to their specific needs, such as in CoCoRoSim [9] or in [10], [11].

Similarly, the general multi-robot and swarm fields rely heavily on simulation: as noted in [12], a large portion of swarm publications are simulation-based. A few swarm simulators are available, such as NetLogo [13], but their application for the underwater can be limited, due to the high level of abstraction necessary to handle huge number of elements. [14] presents an analysis of simulators available for underwater multi-agents.

As the focus transitions from single vehicle to multi-robot and swarm interaction, the single agent low-level control becomes less relevant; this allows us to approximate the closed-loop control response of each agent, such as depth and heading keeping, in the resolution of the agent motion. It is however still useful to maintain a certain level of fidelity on planar motion, current disturbances and acoustic communication, as those can directly affect multi-robot interaction underwater. Motivated by the development of the H-SURF project (Heterogenous Swarm of Underwater Robotic Fish [15], [16]), we propose a simulation environment for multi-agent underwater vehicles: SwarmSwIM. SwarmSwIM is designed to operate on a lightweight core that allows for different level of abstraction, with on-demand plug-ins such as 2D visualization or 3D rendering, acoustic communication with Doppler simulation, and relative detection. The simulator is available on <https://github.com/save-xx/SwarmSwIM>.

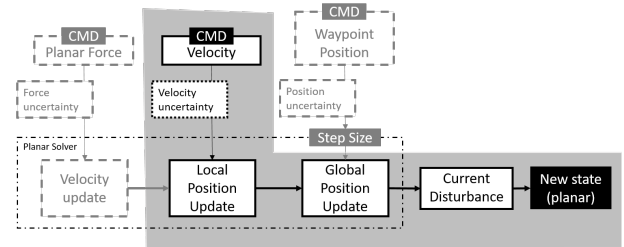


Fig. 1: Schematic of planar position update for one agent. Highlighted the pipeline for the velocity control case.

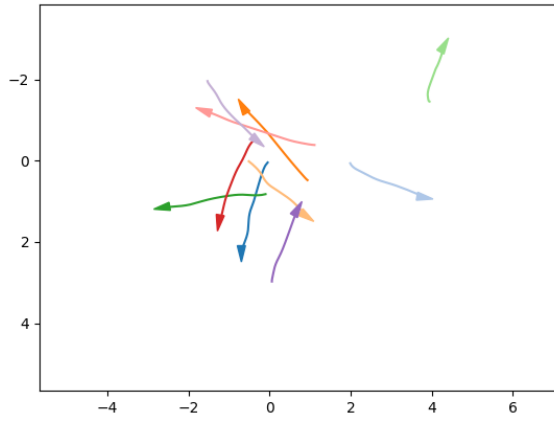


Fig. 2: Animation 2D plug-in: output with multiple agents position and heading represented on a plane, moving on straight trajectories under influence of currents.

II. METHODS

The simulation is designed to enable the emulation of multi-robot behaviors underwater. Each member of the swarm is identified with the name *agent*. The core principle is the simplification of the models driving the motion of the agent, by substituting the full dynamic simulation with an informed approximation driven by the desired or expected behavior of the controller acting on the agent itself. The simulator generates an instance for each agent. An agent's state is determined by four DoF: 3D position and yaw, while pitch and roll angles are neglected, as they usually have a minimal impact on the multi-agent global trajectories and behaviors. The simulation is based on a fixed-time subdivision and can be run either in Real-Time (RT) or faster than RT for applications such as machine learning. Each agent is assigned a command subdivided into planar, vertical, and heading. The output of the sensors (both interoceptive and exteroceptive) is also emulated, based on the characterization of the sensors accuracy and precision. In particular, emulated sensor output is used in the simulator's control loop to model the uncertainty effects on the resulting trajectories. This approach allows to easily upscale both the volume of the simulation environment and the number of agents, maintaining a reasonable computational cost. It also has the advantage of allowing abstraction of the agent architecture and control scheme, which grants the development of behaviors in simulation without a precise knowledge of the agent's physics and dynamics. This simulator has been extensively used for the design of distributed control methods in [17], where it particularly demonstrated the feasibility of acoustic-related behaviors in underwater swarms prior to deploying the robots in the sea.

A. Simulator Core

The core of the simulator deals with updating both agents and environment, and keeps track of the most recent agent states. It constitutes a basis for the simulation of each agent sensor (state dependent) and other common underwater tools

(environment dependent), providing a global knowledge of the system's status. A description of the external plugins will be carried on in II-D. We can separate the main cycle of the simulator in two logic functions: *simulation tick* and *agent tick*. The *agent tick* is related to the state update of each agent for each interval Δt . Firstly, each agent updates its internal sensors' status, i.e. the measured state. The measured state is related to the adopted controls and includes: planar position, velocity, depth and heading. In the computation of the measured state, a gaussian error is introduced to simulate imprecision of the sensors involved in the feedback of the emulated control scheme. This is followed by the model dynamics update, that strictly depends on the control method, which will be discussed in II-B.

The *simulator tick* wraps all the *agents tick*, and includes several other operations such as current disturbance and state logging. The tick is also responsible of keeping the time. The latest locations of the agents are traced to compute time-dependent measurements, such as acoustic ranging or Doppler velocity estimation.

Since sensor feedback measurements and certain current models (described in II-C) incorporate randomness, which directly influences the system's state evolution, the simulator employs a pseudo-random number generator (PCG64 from the NumPy library). By default, a new random seed is assigned at each run. However, users can specify a global *seed* value at the simulation level to initialize the generator with a fixed seed. This ensures deterministic pseudo-random sequences, guaranteeing repeatability — an essential requirement for full simulation reproducibility.

The simulation uses two types of *xm1* file to set all the user defined parameters (see III). A single simulation *xm1* file defines the global currents, the number and initial position of the agents and the random seed. An agent file describes the general proprieties of an agent type, hydrodynamic parameters, sensor feedback uncertainties and control scheme parameters. For agents with different proprieties, multiple *xm1* files can be included, each with the description of the different agent type proprieties.

B. Vehicle Control Options

The core principle of the simulator is to approximate the system dynamics by predicting the agents behavior based on the applied control, rather than performing a full dynamic estimation and associated low level control. This prediction is based on the type of control that is assumed for each agent and each degree of freedom. The first assumption made is the decoupling of planar moment, vertical (heave) movement and heading rotation. For each one of the three, the simulator provides a set of control schemes.

1) *Depth*: Depth control assumes a feedback on the current depth or the depth rate (velocity of ascend or descend). This is a common feedback of underwater systems based on the pressure reading. For a direct depth measurement, the desired depth is compared to the emulated measurement, to obtain the control error. When the control method is set to *ideal*, the

control error is added to the current state (moving the agent immediately to the desired depth, plus measurement error). When the *step* option is enabled, the maximum correction for each step of simulation is determined by a fixed velocity defined by the user. The *proportional* control emulates a simple proportional controller that uses the control error as input and outputs the vertical velocity. Such velocity is then integrated in Δt to correct the vertical position. Finally the *heave speed* takes a vertical velocity as input, applies Gaussian noise and integrate as in *proportional*.

2) *Heading*: Heading control assumes that the agent use feedback to correct and control their heading. Typically gyroscope and magnetometers provide readings to derive heading and yaw rate. Similarly to depth control, control error is obtained comparing the measured heading and desired heading in a $[-\pi, \pi]$ interval. The same schemes are available for the heading control as in depth: *ideal*, *step*, *proportional*, and for direct angular velocity control *rotation rate*. The agent heading is relevant for controls based on body axis, such a control acting on surge. As such, the heading correction is done before the planar correction.

In table I a sum-up of the equations governing the control laws of depth and heading.

TABLE I: Depth and Heading Control Options

Control (var)	Heading	Depth
<i>Ideal (pos)</i>	e_{dep}	e_{ψ}
<i>Step (pos)</i>	$\min(\text{step}_{dep}, e_{dep})$	$\min(\text{step}_{\psi}, e_{\psi})$
<i>Proportional (vel)</i>	$\text{SAT}_{w_{max}}(k_p e_{dep})$	$\text{SAT}_{\omega_{\psi, max}}(k_p e_{\psi})$
<i>Speed (vel)</i>	w	ω_{ψ}

3) *Planar update*: The update on the horizontal plane follows rules similar to the one presented in the previous cases.

The *ideal* positional control moves the agent directly to the desired (x, y) position in one step, while the *step* controller updates a maximum distance d on the line that connects the current position to the desired one (way-point control).

A higher-level controller is the *local velocity* controller, that updates the current position based on a velocity command expressed in the body-frame reference system. This allows control of the velocity of the agent with respect of the body of water, upon which are added the effects of currents. This control emulates a model-based estimation of the velocity and/or dead-reckoning, where the resulting velocity is combination of actuation and currents. Alternatively, the *inertial velocity* controller directly regulates the agent's velocity, independently of the surrounding currents. A predefined "maximum effort" constrains the agent's ability to counteract currents, effectively setting an upper limit on its achievable velocity. This approach emulates feedback from an Inertial Navigation System (INS) or a Doppler Velocity Log (DVL), allowing the agent to maintain a desired global velocity despite current disturbances—up to the point where the disturbance

exceeds the agent's maximum thrust capacity. Finally, the *local forces* controller takes into account the hydrodynamic behavior of an underwater agent to compute the agent's acceleration, and then, consequently, uses Euler semi-implicit integration method to obtain speed and position. In table II a sum-up of the different control laws for the planar case.

TABLE II: Planar Position Control Options

Control (var)	Law
<i>Ideal (pos)</i>	$[e_{pos_x}, e_{pos_y}]^T$
<i>Step (pos)</i>	$\min(\text{norm}(e_{pos}), \text{step}_{pos}) \hat{e}_{pos}$
<i>Local Velocity (vel)</i>	$R_z(\psi) \boldsymbol{\nu}$
<i>Inertial velocity (vel)</i>	$R_z(\psi) \min(\mathbf{v}, \boldsymbol{\nu}_{max})$
<i>Local Forces (acc)</i>	$M^{-1}(\tau + X_{\nu} \boldsymbol{\nu} + X_{\nu\nu} \boldsymbol{\nu} \boldsymbol{\nu})$

Where $M = m\mathbf{I} + M_a$ is a 2×2 matrix representing the mass and added mass, $\boldsymbol{\nu}(u, v)$ the planar velocity, respect to the body of water and oriented in body axis, while $\mathbf{v}$ is measured respect to ground. e_{pos} represents the difference between current measured position and desired position.

C. Current and Disturbances

The simulator implements a method to introduce the currents disturbances. The latest version focuses entirely on planar currents, with no gradient or variation with depth. The current disturbance is applied as a perturbation of the planar position of each agent, applied at the beginning of each simulation step. Each agent is affected based on its current location, and time. It however does not consider, in the latest version, differences due to different size or geometries for different agent types. A set of different type of currents and disturbances are available and can be combined: uniform, global waves, local waves, noise, vortexes. Each method functions by calculating, for each simulation step and agent, a disturbance velocity $\mathbf{v}_d$, which is multiplied by the time step to become a spatial disturbance and applied to the agents positions:

$$\mathbf{p}_i(x, y) \leftarrow \mathbf{p}_i(x, y) + \mathbf{v}_d(x, y) \Delta t \quad (1)$$

Where $\mathbf{p}_i(x, y)$ is the planar position of the i -th agent. In fig. 3, two examples of representation of the disturbances resulting fields are showcased. It is worth noting that in actual simulation the current is only calculated in the location of the agents, thus making the method unaffected by the volume of the simulation.

Following the description of each current model:

1) *Constant currents*: The velocity disturbance $\mathbf{v}_d$ is defined by a constant vector, equal for all agents.

2) *Global waves*: The velocity disturbance $\mathbf{v}_d$ is defined as:

$$\mathbf{v}_d = A \sin(2\pi f t + \phi) \cdot \hat{\mathbf{u}} \quad (2)$$

Where amplitude A , versor of direction $\hat{\mathbf{u}}$, phase ϕ and frequency f are decided by the user and t is time. The current is applied equally among all agents in the simulation. Multiple

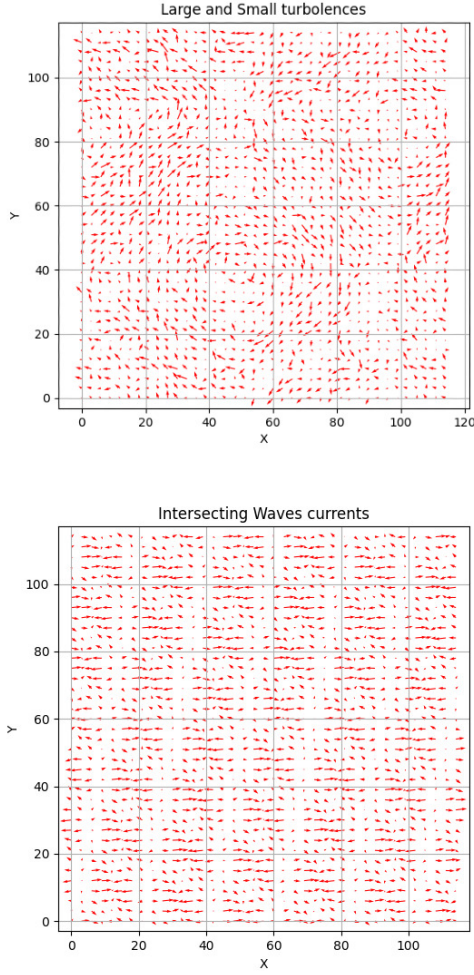


Fig. 3: Examples of current fields affecting the agents: Above, combination of small time-dependent and large time-independent turbulent-like, generated for an instance of time combining noise and vortex currents models. Below, intersecting wave currents with different wavelength, direction and intensities, modeled by application of two local waves currents.

instances can be used at once. This disturbance represents a periodic variation over time affecting the whole field.

3) *Local waves*: The local waves velocity disturbance $\mathbf{v}_{d_i}$, for each agent is defined as:

$$\mathbf{v}_{d_i} = A \sin[2\pi w_l(w_s t + \mathbf{p}_i \cdot \hat{\mathbf{u}}) + \phi] \cdot \hat{\mathbf{u}} \quad (3)$$

Where w_l, w_s are the wave length (distance between 2 peaks) and wave-speed. The model limits the direction of the disturbance to the propagation direction of the wave. Each agent is affected differently, based on its planar position. Multiple instances can be used at once. Note that the amplitude and wave-speed are distinct values. This disturbance represent a structured wave-like disturbance in time and space. This model closely resembles the model proposed in [18].

4) *Noise*: The noise disturbance $\mathbf{v}_{d_i}$, for each agent is defined as:

$$\mathbf{v}_{d_i} = f[1 - (t - t_K)]\mathbf{v}_{r,K} + f[t - t_{K+1}]\mathbf{v}_{r,K+1} \quad (4)$$

Which is a linear interpolation of two vectors $\mathbf{v}_{r,K}$ and $\mathbf{v}_{r,K+1}$, referring to time instances t_K in the past and t_{K+1} in the future, at time t . The t_K instances are evenly spaced at frequency f . The vector $\mathbf{v}_{r,K+1}$ is calculated as:

$$\begin{aligned} \mathbf{v}_{r,K+1} &= \mathbf{v}(r, \theta) \\ r &\sim U(0, A) \\ \theta &\sim U(0, 2\pi) \end{aligned} \quad (5)$$

Where the two polar components of the vector are obtained from two uniform distribution in the range $(0, A)$ and $(0, 2\pi)$ respectively. When the simulation time t reaches t_{K+1} , $\mathbf{v}_{r,K}$ becomes $\mathbf{v}_{r,K+1}$ and a new $\mathbf{v}_{r,K+1}$ is generated. This disturbance emulates localized time dependent disturbances and small vortexes.

5) *Vortexes*: The vortex disturbance is a space-dependent and time-independent disturbance. It is calculated as the sum of the contribution of each vortex to a point in space. A user defined number N of vortex centers is initially generated and distributed in a 100 x 100 m area. The same points are virtually tiled with a period of 100 m in all directions. Given a generic agent in a time instance, all vortex points in a 50 meter distance from the agent on either axis are used to calculate the disturbance. A single vortex contribution for a position is obtained as follows (excluding agent index for clarity):

$$\mathbf{v}_j = \frac{A_j}{(\|\mathbf{p} - \mathbf{c}_j\|^2 + 1)^{0.75}} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} (\mathbf{p} - \mathbf{c}_j) \quad (6)$$

where $\mathbf{p}$ is the agent planar position and $\mathbf{c}_j$ is the position of the j -th vortex. This disturbance emulates the effect of large vortexes and persistent disturbances of the current flow (such as variation due to not uniform seabed).

D. Plugins

1) *Visual Detector Sensor*: This plugin emulates a visual detector mounted on each agent, designed to identify and localize other agents within its field of view. It follows the same principle as the methodology presented in [19], which employs a CNN to estimate the relative positions of detected agents. However, the plugin is adaptable to other relative detection methods. At each simulation step, the plugin can be triggered for any agent, according to each predefined sensor update rate, which is specified by the user. Upon activation, the plugin determines which agents fall within the field of view (FoV), assesses whether detection is successful, and converts the detected relative positions into spherical coordinates. This transformation better approximates the data obtained when estimating 3D positions from 2D images. Gaussian noise and bias are then applied to simulate realistic sensor inaccuracies. The FoV, detection success rate, and measurement noise are all user-configurable parameters, allowing customization based on the characteristics of the simulated sensor. Notably, the detection success rate can be defined as a function of relative

distance, specified by discrete points. A typical function starts at zero at very close range, reaches a peak at an optimal distance, and then declines back to zero as distance increases, mimicking the decreasing reliability of visual detectors at greater distances.

2) *Acoustic channel*: The acoustic plugin emulates the propagation of acoustic based communication between modems. The user can define one or more acoustic channels. Within the channel, any agent can initiate a communication with a set packet time span. The plugin emulates the spherical propagation of the message, identifies concurrent reception (collisions), and based on the instances of “start sending” time and “start reception” time, calculates the time of flight of the message, which consider the agents internal clock, affected by measurement drift. Furthermore, using the average velocities of the two agents, the average rate-of-range is calculated, as well as the average of the projection of the velocities difference, computed along the direction defined by their positions. Specifically:

$$v_d = (\mathbf{v}_r - \mathbf{v}_s) \frac{\overrightarrow{RS}}{\|\overrightarrow{RS}\|} \quad (7)$$

Where $\mathbf{v}_r, \mathbf{v}_s$ are the respective velocities of the receiver and sender and $\overrightarrow{RS}$ is the vector connecting the receiver to the sender. The v_d emulates the direct measurement that can be obtained by analyzing the Doppler shift of an incoming acoustic message. Finally a bytearray can be specified as payload of the message. The propagation assumes single path and a homogeneous medium with straight paths from agent to agent. This plugin favors studies relevant to the use of acoustic for relative localization, as well as strategies for multi-agents MAC access.

3) *3D rendering interface*: A 3D representation of the simulation has been implemented with Unreal Engine 5, a powerful tool that can extend this simulator capabilities. Simulating a 3D environment allows the inclusion of sensors that are environment-dependent, such as a bathymetry sensor that is subject to the seabed’s morphology, and a visual sensor that can make images available to actual neural networks, in order to detect features (i.e. plants, animals) in the agent’s surroundings.

The most important realism that needs to be introduced in a simulated underwater condition is the light absorption and attenuation system. This has been done with a post-process volume that acts under the sea level (zero depth). The light attenuation is generated using a deepness-dependent *lerp* operation. The color detected by any agent or camera actor in the environment is shifted from a light blue for shallow waters to a dark blue for the sea depths. The visible light absorption is related to the scene depth. Red, Green and Blue color are subsequently dampened from the scene at different distance thresholds, resulting in a complete black background after the last threshold. The two effects are eventually combined. A sample of the result is shown in fig. 4, both from the user’s and the agent’s point of view.

An API has been developed to allow communication between the rendering and the simulator through the HTTP protocol. As example: first, the agents’ position are updated from the simulator in the engine; with the new obtained location, each agent sends back its camera view (JPEG compressed) and its bathymetry data, as well as any other additional sensor.

4) *ROS2 API*: The ROS2 API allows the integration of all the previously described plugins with any custom developed ROS architecture. This extension converts all the generated data into ROS topics, using only standard ROS messages for an easily distributable implementation.

The integration of ROS2 with other plugins strictly depends on ROS synchronization model. All tasks, including ROS spin method, must be gathered in a single pool of events, that are executed concurrently as for the ROS nodes’ execution scheduling system.

There are three main modes of usage for this package:

- **ROS2 Real Time**: this method is able to reproduce a real time computation at a defined frequency, using the internal ROS clock. It is limited by the computational cost of the custom ROS network connected.
- **ROS2 and UE5**: it works as API for Unreal Engine in order to provide a 3D Real Time rendering as well. It also executes the simulation at real time speed, provided sufficient computational power.
- **ROS2 Step Update**: a custom way of implementing the update by sending a flag, the *step* topic (bool). When a new message is received on the *step* topic, the API execute a single simulation step. This is useful, for example, for implementing machine learning (ML) techniques or large systems of agents and controls. On one side, it allows to ensure that the whole simulation-agent logic is executed on the user side, before advancing in the simulation, especially when running the control on numerous agent on a single machine. On the other, when the simulator is used to generate ensemble or the training data for ML, it is convenient to process the simulation without pauses and minimize the processing time required.

III. DEFINING A SIMULATION

In this chapter a description of the `xm1` setup files for simulation and agents will be carried on.

A. Platforms characterization

The parameters to be defined for each different agent type are divided in two sections: *agent* and *sensors*.

In the *agent* section, the physical parameters are defined: mass, added mass matrix, quadratic and (optionally) linear damping. Furthermore, for each vehicle control option (II-B), the mode of control as well as the necessary control parameters are settled. Eventually, the random error parameters (bias and standard deviation) are defined for each element of the internal status sensor, that includes positional and velocity data: depth/heave, heading/yaw-rate, planar position/local velocity.

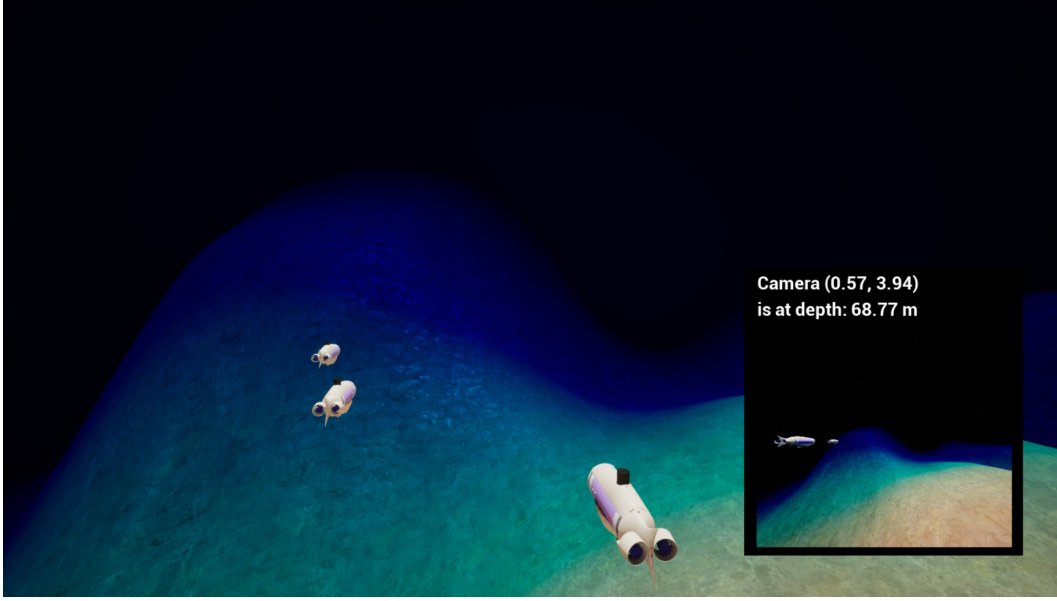


Fig. 4: 3D rendering plug-in in Unreal Engine 5. Deep water. Note on the bottom-right the first person view of the rear agent.

The value simulating the drift of the internal clock (PPM) respect to the real time is also included.

The second section is dedicated to the *sensors* that are loaded in each simulated robotic platform. For each sensor, the average update period and the errors on the estimated values must be defined, besides the sensor parameters. It is provided as example the mainly used sensor for the H-SURF Project, with some extra parameters:

- Neural Network Detector: this is a camera based sensor, so FOV it is necessarily described. The update period mainly depends on the computational power (CPU, GPU) of the platform itself in dealing with neural networks. Due to the typical underwater visibility limitations, a distance-based detection probability function simulates visibility loss. The outputs are the spherical coordinates of the detected object (distance, polar angle, azimuthal angle), with error.
- Acoustic Range Detector: parameters of the localization estimation based on acoustic communication are described. For reference, the *Succorfish Dephi V3* sensor had an single package transmission time of approximately 1 s. The outputs of this sensor are the acoustic range and the relative range-rate measured using Doppler shift.

B. Environment and Setup

The simulation `xml` file describes all the environmental characteristics and all the agents allocated in the simulation. Two main section can be found: *environmental setup* and *agents*.

In the first section, all the different options for simulated currents are included, with their own parameters, as presented in II-C. Specifically the file allows to set to the the following:

- uniform currents: $\mathbf{v}_d$ as North, East components
- noise currents: frequency (Hz), average velocity A

- vortex currents: density N , average velocity σ_v , ($\mathbf{A}_j \sim \mathcal{U}(-\sigma_v, \sigma_v)$)
- global sinusoidal waves: Intensity A , frequency f , and phase ϕ . The $\hat{\mathbf{u}}$ vector is passed as a direction in degree from the North. Multiple instances can be defined
- local sinusoidal waves: Intensity A , wavelength w_l , wave-speed w_s , and phase ϕ . The $\hat{\mathbf{u}}$ is passed as a direction in degree from the North. Multiple instances can be defined

In the second section we can find all the different agent types that needs to be loaded in the simulation. For each agent type, a different descriptor is included (see III-A) and a different name-space is given. Furthermore, a state initialization group is included, where each line represent a single robot instance and provides the main parameters to initialize the state, i.e. position (x, y, z) and heading (ψ angle). Multiple type of agents can be defined in the same simulation, and multiple instances of the same agent, referring to the same file, can be used. The agents are all initialized with no initial velocity, in the position and heading indicated. Agent of the same type use the same name-space, with an additional numerical counter. Finally the optional seed field allow to set an arbitrary integer to be used in the whole simulation random sequential generator, allowing to repeatability of the results. If not defined, a random seed will be used instead.

IV. TESTING

A. Setup

To prove the validity of the described simulator, a comparison with experimental data is presented. The experimental test was conducted in a $8 \times 12 \times 2.2$ m pool. Two agents (platforms described in [15]) are placed in the pool, the former acting as a leader, manually controlled by an operator [20], the latter being completely autonomous in following the leader. The

developed algorithm is based on the *Pure Pursuit* guidance method, that concurrently corrects heading and relative desired distance of the follower. The sensor involved is a RGB camera, and the relative positioning information is generated with a neural network-based image analysis. More information about the test methodology and the obtained results are abundantly described in [19]. The testing facility is equipped with an Optitrack tracking system, placed above the pool. Each agent mounts a hydrofoil 30 *cm* fin that allows the reflective markers to be constantly above water. The data of the Optitrack system is then shared on the ROS message network. This allows a precise time synchronization between the agents' and the tracked data's timestamps prior to the start of the mission.

The data collected from the experimental tracking were used to initialize the simulation: initial position and heading are extracted at a initial state, where the AUVs had almost no planar velocity. The Leader is implemented using the *ideal* planar control, and directly fed the position of the tracker. The follower instead uses the *Visual Detector Sensor* Plugin to determine the emulated visual based information on its relative position from the Leader. It then implements a bare-bone simplified version of the Leader-Follower control described in [19], and returns a surge-trust and heading command. The simulators runs for a time windows of 83 seconds to capture the showcased path. The estimation of the AUV parameter used are a rough estimation of the one used in the experiment, with the most relevant values reported in Table III. Finally, a time subdivision of $\Delta t = 0.05s$ was set.

TABLE III: Simulated Agent parameters

Mass	4.4 Kg
Added Mass - diagonal	1.0, 2.0 Kg
Quadratic Damping - diagonal	-6.0, -12.0
Heading Control	<i>step</i> , 15.0°
Planar Control	<i>local forces</i>
Visual horizontal FoV	120°
Visual detection period	0.4 seconds

An ensemble of another 50 iteration of the simulation has been performed to visualize the combined impact of the most common uncertainties. A weak *noise current* has been added and errors on the heading and detection estimation have been added, with varying initial measurement biases (sampled from a uniform distribution). In Table IV are reported the parameters used in the ensemble.

B. Results

The simulation generates a estimated path of the Follower, given a known Leader path. In fig. 5 it is shown the Leader path, the effective Follower path as recorded experimentally, with application of the control model presented in [19], the simulated path, with no uncertainties added, and the ensemble of simulated result, affected by the noise and uncertainties

TABLE IV: Simulated Agent and environment noise and uncertainties

Detection Probability- Interpolated	0% (0.3m) 95% (0.4-0.2m) 60% (5m) 0% (6m)
Detection range error - meters	± 0.05 , $\sigma = 0.15$
Detection range error	$\pm 0.5^\circ$, $\sigma = 1^\circ$
Heading error	$\pm 1.0^\circ$, $\sigma = 2.0^\circ$
Noise current	Intensity 0.04m/s, 1Hz

described above. Figure 6 shows the difference between simulated and tracked paths, which results to a mean error of 0.034 *m* for the simulation and an average 0.05 *m* for all the ensemble. The qualitative result showcases that, despite the numerous approximations introduced, the simulation is able to capture quite closely and predict the final behavior of the system. The detailed simulation of the noises and errors allows to study how impactful they can be on a desired behavior and control scheme. The case presented is qualitatively robust to variation and disturbances, with all the ensemble results falling closely to the original simulation and experimental result. This test is an initial validation of the predictive capabilities of the simulator. The error and noise simulator allows to study the tested behaviors robustness and the impact of each distinct source of error independently or combined.

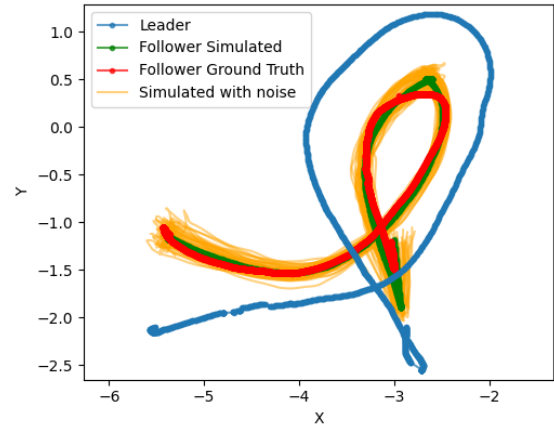


Fig. 5: Comparison of experimental testing path tracking, with simulated result. In Picture are represented the paths of the leader and follower AUVs on the plane, and the simulated path of the follower. In background are plotter and ensemble of simulation results with currents and sensor noise and perturbations.

V. ACKNOWLEDGMENTS

The authors acknowledge the project “Heterogeneous Swarm of Underwater Autonomous Vehicles” funded by the Technology Innovation Institute (Abu Dhabi) developed with

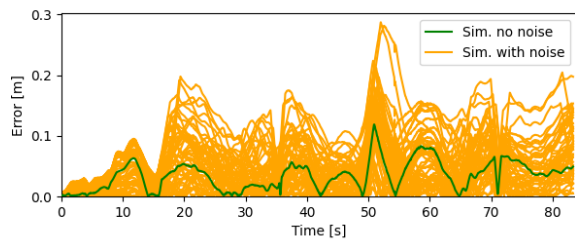


Fig. 6: Path difference in meters over time of the tracked Follower position compared to simulated one. In yellow tracked path to ensemble path differences.

Khalifa University (contract no. TII/ARRC/2047/2020), the "RoboGenAI" project, funded by Dubai Future Foundation, and the Khalifa University Center for Autonomous Robotic Systems (KUCARS).

REFERENCES

- [1] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *2004 IEEE/RSJ international conference on intelligent robots and systems (IROS)(IEEE Cat. No. 04CH37566)*, vol. 3. Ieee, 2004, pp. 2149–2154.
- [2] robodk, "robodk," <https://robodk.com/>, accessed: 2024-11-22.
- [3] M. M. M. Manhães, S. A. Scherer, M. Voss, L. R. Douat, and T. Rauschenbach, "UUV simulator: A gazebo-based package for underwater intervention and multi-robot simulation," in *OCEANS 2016 MTS/IEEE Monterey*. IEEE, sep 2016. [Online]. Available: <https://doi.org/10.1109%2FOceans.2016.7761080>
- [4] B. Bingham, C. Agüero, M. McCarrin, J. Klamo, J. Malia, K. Allen, T. Lum, M. Rawson, and R. Waqar, "Toward maritime robotic simulation in gazebo," in *OCEANS 2019 MTS/IEEE SEATTLE*. IEEE, 2019, pp. 1–10.
- [5] M. Prats, J. Perez, J. J. Fernández, and P. J. Sanz, "An open source tool for simulation and supervision of underwater intervention missions," in *2012 IEEE/RSJ international conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 2577–2582.
- [6] dynautics, "dynautics auv-sim," <https://www.dynautics.com/products/vessel-simulators/auv-sim/>, accessed: 2024-11-22.
- [7] A. Luvisutto, A. A. Shehhi, N. Mankovskii, F. Renda, C. Stefanini, and G. De Masi, "Robotic swarm for marine and submarine missions: Challenges and perspectives," in *2022 IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, 2022, pp. 1–8.
- [8] E. Rohmer, S. P. N. Singh, and M. Freese, "Coppeliasim (formerly v-rep): a versatile and scalable robot simulation framework," in *Proc. of The International Conference on Intelligent Robots and Systems (IROS)*, 2013, www.coppeliarobotics.com.
- [9] M. Read, C. Möslinger, T. Dipper, D. Kengyel, J. Hilder, R. Thenius, A. Tyrrell, J. Timmis, and T. Schmickl, "Profiling underwater swarm robotic shoaling performance using simulation," in *Towards Autonomous Robotic Systems: 14th Annual Conference, TAROS 2013, Oxford, UK, August 28–30, 2013, Revised Selected Papers 14*. Springer, 2014, pp. 404–416.
- [10] Q. Jia, H. Xu, X. Feng, H. Gu, and L. Gao, "Research on cooperative area search of multiple underwater robots based on the prediction of initial target information," *Ocean Engineering*, vol. 172, pp. 660–670, 2019.
- [11] S. P. Hou and C. C. Cheah, "Can a simple control scheme work for a formation control of multiple autonomous underwater vehicles?" *IEEE Transactions on Control Systems Technology*, vol. 19, no. 5, pp. 1090–1101, 2010.
- [12] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: a review from the swarm engineering perspective," *Swarm Intelligence*, vol. 7, pp. 1–41, 2013.
- [13] U. Wilensky, "Netlogo," Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, <http://ccl.northwestern.edu/netlogo/>, 1999. [Online]. Available: <http://ccl.northwestern.edu/netlogo/>
- [14] D. Cook, A. Vardy, and R. Lewis, "A survey of auv and robot simulators for multi-vehicle operations," in *2014 IEEE/OES Autonomous Underwater Vehicles (AUV)*. IEEE, 2014, pp. 1–8.
- [15] S. Iacoponi, G. J. Van Vuuren, G. Santaera, N. Mankovskii, I. Zhilin, F. Renda, C. Stefanini, and G. De Masi, "H-surf: Heterogeneous swarm of underwater robotic fish," in *OCEANS 2022, Hampton Roads*. IEEE, 2022, pp. 1–5.
- [16] S. Iacoponi, M. Hanbaly, A. Infanti, B. Andonovski, N. Mankovskii, I. Zhilin, F. Renda, C. Stefanini, and G. De Masi, "Heterogeneous underwater swarm of robotic fish: Behaviour and applications," in *to appear in International Conference on Embodied Intelligence*. IOP, 2023.
- [17] M. Tarnini, S. Iacoponi, A. Infanti, C. Stefanini, G. De Masi, and F. Renda, "Boundary control behaviors of multiple low-cost auvs using acoustic communication," *pre-print*, 2024, arXiv:2412.00118.
- [18] D. Chang, X. Liang, W. Wu, C. R. Edwards, and F. Zhang, "Real-time modeling of ocean currents for navigating underwater glider sensing networks," *Cooperative robots and sensor networks*, pp. 61–75, 2014.
- [19] A. Infanti, S. Iacoponi, S. Alameri, M. El Hanbaly, M. Tarnini, C. Stefanini, G. De Masi, and F. Renda, "Formation control with obstacle avoidance of underwater swarms based on relative visual feedback," 2025, under review.
- [20] S. Iacoponi, N. Mankovskii, M. El Hanbaly, A. Infanti, S. Alhajeri, F. Renda, C. Stefanini, and G. De Masi, "Wireless teleoperation of hsurf artificial fish in complex paths," in *OCEANS 2023-Limerick*. IEEE, 2023, pp. 1–5.